SHIV NADAR UNIVERSITY

Programme: **B.Tech**
Exam: **Mid Semester**
Course Code: **CSD201**
Date: **February 27, 2019**

Discipline: **Computer Science and Engineering**
Year: **2018-2019**
Course Title: **Data Structures**
Time: **10:00 AM – 11:30 AM** Max. Marks: **40**

Ques.1 If a complete binary tree has 'n' leaves and all levels are fully populated, how many nodes does the tree have in terms of 'n'? $2n - 1$ **(1 Mark)**

Ques.2 A solution to a 64-disk Tower of Hanoi problem requires $2^{64} - 1$ moves. **(1 Mark)**

$O(n^2)$ **Ques.3** Given an n-element sequence S, **Algorithm D** calls Algorithm E on each element S[i]. **Algorithm E** runs in O(i) time when it is called on element S[i]. What is the worst case running time of **Algorithm D**? **(1 Mark)**

Ques.4 Convert the following Infix Expressions into equivalent postfix Expressions. Show the content of stack also during conversion. **(2 Marks)**

(a) (A && B) || C || ! (E > F) A B && C || E F > ! ||

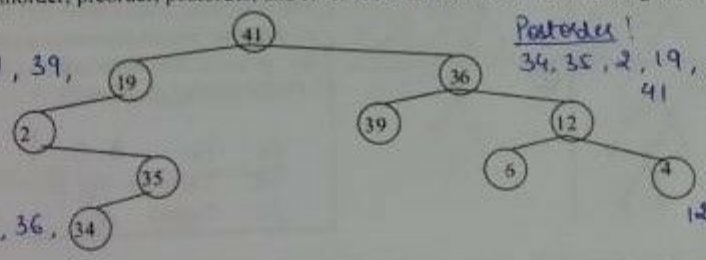
(b) ! (A && ! ((B < C) || (D > E))) || (F < G) A B C < D E > || ! && ! F G < ||

(Consider the operator's precedence same as studied in ICP)

Ques.5 What is the minimum height of a binary tree having 'N' nodes. Prove your answer. **(2 Marks)**

$\log_2(n+1)/2$ (Refer Slide No. 30 of Trees & Binary Trees).

Ques.6 Give the inorder, preorder, postorder, and level order traversals for the following tree. **(2 Marks)**



Inorder:
2, 34, 35, 19, 41, 39, 36, 6, 12, 4

Preorder:
41, 19, 2, 35, 34, 36, 39, 12, 6, 4

Postorder:
34, 35, 2, 19, 39, 6, 4, 12, 36, 41

levelorder:
41, 19, 36, 2, 39, 12, 35, 6, 4, 34

Ques.7 Given an n-element sequence S of integers, **Algorithm C** executes an O(n)- time computation for each even number in S, and an O(log n)- time computation for each odd number in S. What are the best case and worst case running times of Algorithm C? **(2 Marks)**

Best Case – $O(\log n)$
Worst Case – $O(n^2)$

Ques.8 There exists a single binary tree with 11 nodes. Below are the postorder and inorder traversals of the tree. Draw the tree. (There is only ONE tree that has BOTH these traversals.) Attempt to deduce your answer in a systematic (and recursive) fashion, not by trial-and-error methods. **(2 Marks)**

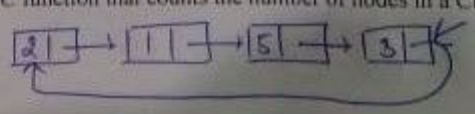
Postorder: A B C D E F I K J G H **Inorder:** C B A E D F H I G K J

Ques.9 Suppose, we have a linked list that:

- Is an ***unordered circular linked list*** with no dummy nodes
- Its nodes are singly-linked (have only a next pointer), and
- The linked list has only a ***last*** reference which points to the last node in the linked list

(a) Draw the linked list storing the data elements [2, 1, 5, 3], where 2 is the first element in the list and 3 is the last element in the list. Make sure you show the links and what node is referenced by the variable ***last***. **(1 Mark)**

(b) Implement a C function that counts the number of nodes in a Circular Linked List. **(3 Marks)**



Ques.10 Find the Big-Oh time complexity of the following code segments: **(3 Marks)**

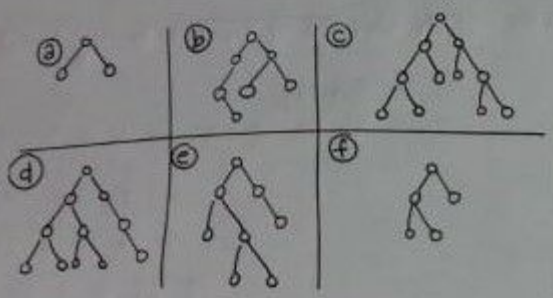
(a)
```
sum = 0, j = -1, i;
for(i = 0; i < n; i++)
{
if ( i > j)
        sum = sum + 1;
else
{
for(k = 0; k < n; k++)
        sum = sum - 1;
}    }
```
O(n)

(b)
```
void smiley (int n, int sum)
{
for (int i = 0; i < n * 100; ++i)
{
for (int j = n; j > 0; j--)
        sum++;
for (int k = 0; k < i; ++k)
        sum++;
}
}
```
$O(n^2)$

O(1) or O(3)

(c)
```
int x = 0, i;
for(i = n; i > = 0; i--)
{
        if((i % 3) = = 0)
                break;
        else
                x + = i;
}
```

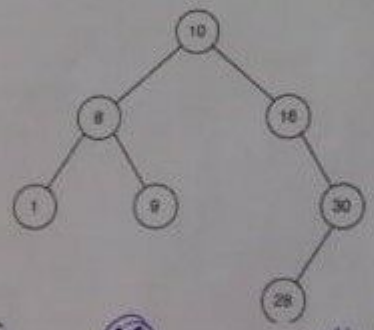
Ques.11 Given the following six trees a through f: **(3 Marks)**



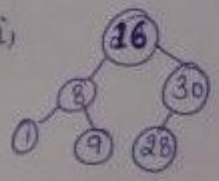
List the letters of all of the trees that have the following properties: (Note: It is possible that none of the trees above have the given property, it is also possible that some trees have more than one of the following properties.)

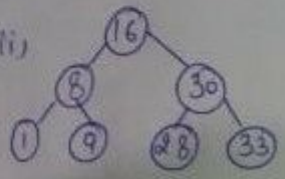
(i) Full a, c, f
(ii) Complete a, f
(iii) Perfect a

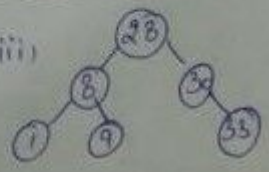
Ques.12 Given the following BST. Draw the binary search tree after each of the following operations. (Operations are cumulative.) Assume the remove operations on nodes with two children take a replacement node from the right child's subtree. **(3 Marks)**



(i) Remove 10.
(ii) Add 33.
(iii) Remove 16

(i)


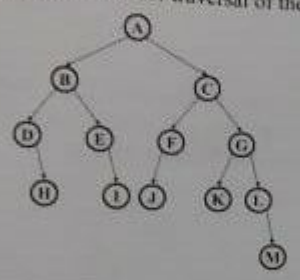
(ii)


(iii)


```
void back_inorder (node *root)
{   if (root == NULL)
        return;
    if (root->right != NULL)
        back_inorder (root->right);
    printf ("%d", root->data);
    if (root->left != NULL)
        back_inorder (root->left);
}
```

Ques.13 Implement a **recursive C function** to print **Backward Inorder traversal** of a tree. (You may look into the following example to understand the meaning of backward Inorder traversal) **(5 Marks)**

The backward Inorder traversal of the following tree is **M, L, G, K, C, F, J, A, I, E, B, H, D**



Ques.14 What are the time complexities of each of the following tasks? (Choose from O(1), O(logn), O(n), O(n logn), O(n^2), O(2^n), each order may appear more than once.) **(5 Marks)**

(a) Popping an item off a stack containing 'n' items. O(1)
(b) Performing a Towers of Hanoi algorithm with 'n' disks. O(2^n)
(c) Using quicksort to sort an array of 'n' integers, in the worst case. O(nlogn)
(d) Performing a bubble sort on an array of 'n' integers, in the worst case. O(n^2)
(e) Displaying all 'n' elements in a sorted linked list. O(n)

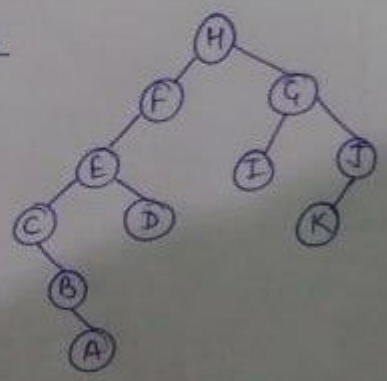
Ques.15 Given the following mystery method: **(4 Marks)**

```
static void mystery(int arr[ ], int L, int H)
{
    if (L >= H)
        return;
    if (arr[L] > arr[H])
    {
        int t = arr[L];
        arr[L] = arr[H];
        arr[H] = t;
    }
    if (H - L + 1 > 2)
    {
        int t = (H - L + 1) / 3;
        mystery(arr, L, H - t);
        mystery(arr, L + t, H);
        mystery(arr, L, H - t);
    }
}
```

and given the following call:

```
int arr[ ] = { 2, 4, 5, 3, 1 };
mystery(arr, 0, arr.length - 1);
```

- Assume that the display method prints out the elements of the array, write out the elements of arr after the execution of the first "if" statement and after the termination of every recursive call. **Hint:** Do not try to trace all recursive calls. Understand the functionality of the program by reading the code carefully.

```
static void mystery(int arr[ ], int L, int H)
{
        if (L >= H)
                return;
        if (arr[L] > arr[H])
        {
                int t = arr[L];
                arr[L] = arr[H];
                arr[H] = t;
        }
        display(arr);                   -----> arr = ?      1, 4, 5, 3, 2
        if (H - L + 1 > 2)
        {
                int t = (H - L + 1) / 3;
                mystery(arr, L, H - t);
                display(arr);           -----> arr = ?      1, 3, 4, 5, 2
                mystery(arr, L + t, H);
                display(arr);           -----> arr = ?      1, 2, 3, 4, 5
                mystery(arr, L, H - t);
                display(arr);           -----> arr = ?      1, 2, 3, 4, 5
        }
}
```

Answer to Q-8